

Name: [Your Name]

Lab Program Number: 10

Date: [Submission Date]

Title: Understanding and Implementing 2D Rotation

Theory

2D Rotation

2D Rotation is a geometric transformation that rotates each point of an object by a given angle θ about a fixed point (usually the origin). This transformation changes the orientation of the object while preserving its shape and size.

Mathematical Representation:

For a point (x,y) rotated by an angle θ (in radians) about the origin, the new coordinates (x',y') are:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

Matrix Representation:

In homogeneous coordinates, rotation about the origin can be represented using a 3×3 transformation matrix:

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x & y & 1 \end{bmatrix}$$

Rotation About an Arbitrary Point:

If rotation is about a point (x_r, y_r) , the transformation includes translation to origin, rotation, and translation back:

1. Translate point so that (x_r, y_r) is at origin: $x_1 = x - x_r$, $y_1 = y - y_r$
2. Rotate by θ : $x_2 = x_1 \cos \theta - y_1 \sin \theta$, $y_2 = x_1 \sin \theta + y_1 \cos \theta$
3. Translate back: $x' = x_2 + x_r$, $y' = y_2 + y_r$

2D Rotation Algorithm:

1. Start
2. Input original coordinates (x,y) , rotation angle θ , and rotation point (x_r, y_r)
3. Apply rotation transformation using the above formulas
4. Update object position to (x',y')
5. Stop

Program for 2D Rotation

```
#include <stdio.h>
#include <graphics.h>
#include <math.h>

void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3) {
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

void rotate(int *x, int *y, float angle, int xr, int yr) {
```

```

float rad = angle * (M_PI / 180.0);
int x_temp = *x - xr;
int y_temp = *y - yr;

*x = xr + round(x_temp * cos(rad) - y_temp * sin(rad));
*y = yr + round(x_temp * sin(rad) + y_temp * cos(rad));
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    int x1 = 150, y1 = 100;
    int x2 = 250, y2 = 100;
    int x3 = 200, y3 = 200;

    // Draw original triangle
    drawTriangle(x1, y1, x2, y2, x3, y3);

    float angle = 45.0; // rotation angle in degrees
    int xr = 200, yr = 150; // rotation point

    // Rotate triangle
    rotate(&x1, &y1, angle, xr, yr);
    rotate(&x2, &y2, angle, xr, yr);
    rotate(&x3, &y3, angle, xr, yr);

    // Draw rotated triangle
    drawTriangle(x1, y1, x2, y2, x3, y3);

    getch();
    closegraph();
    return 0;
}

```

Implementation / Output

Output Description:

The program displays a triangle before and after rotation. The original triangle is drawn in one color, and the rotated triangle is drawn in another. The output clearly labels:

Object: Original triangle

Image: Rotated triangle

Quadrant: Both triangles are shown within the first quadrant, with rotation about a specified point.

(Attach a printout of your graphics output here, with clear labels.)

Conclusion

This lab successfully demonstrated the implementation of 2D rotation in Computer Graphics. By applying a rotation angle θ about a fixed point, the triangle was rotated accurately while maintaining its shape and size. The exercise reinforced the mathematical and matrix representations of rotation and its practical application in graphical transformations. The output clearly showed the change in orientation, confirming the correctness of the implemented algorithm.

References

https://drive.google.com/file/d/1J7BlqF2og-fAk693JfBWdul_Z1F5pb6/view?usp=sharing